



Univerzitet u Beogradu, Fakultet organizacionih nauka
Katedra za elektronsko poslovanje

Domaći I – Projektovanje mikroservisne aplikacije

Web aplikacija za upravljanje i automatizaciju sportskih događaja sa preprekama

Redni broj	Ime i prezime	Broj indeksa
1.	Vukašin Kovačević	2022/0227
2.	Sofija Turina	2022/0478
3.	Bogdan Mrdeljić	2022/0048
Mentor:	Miloš Jolović	
Link do github-a:	https://github.com/elab-development/rnaep-oas-projekat-rnaep_race_mgmt_webapp_2022_0227	

1. Opis problema

Tržište trka sa preprekama (OCR) trenutno je potpuno fragmentisano. Ne postoji centralno mesto koje okuplja organizatore i trkače, već se sve svodi na izolovanu i zastarelu administraciju.

- **Manuelna naplata:** Većina trka i dalje funkcioniše po principu fizičkih uplatnica, gde organizatori ručno proveravaju izvode iz banaka i potvrđuju prijave, što dovodi do haosa u evidenciji.
- **Nedostatak zajednice:** Trkači nemaju uvid u istoriju svojih prijava, rezultate ili pregled svih dostupnih događaja na jednom mestu.
- **Nepouzdanost podataka:** Informacije o trkama se šire preko društvenih mreža, dok su sami procesi prijave „pusto“ popunjavanje formi bez ikakve povratne informacije ili sigurnosti.

ObstaRace menja pravila igre tako što uvodi standardizovanu platformu koja eliminiše manuelni rad i povezuje sve aktere u jedan digitalni ekosistem.

Domen sistema je integrisana platforma za OCR događaje, koja funkcioniše kao spona između organizatora i sportista. Ključni segmenti domena su:

- **Centralizovan Marketplace trka:** Jedinstven pregled svih OCR događaja, gde organizatori mogu da kreiraju trke, a trkači da ih pretražuju i filtriraju.
- **Automatizovan finansijski tok:** Potpuna zamena papirnih uplatnica digitalnim plaćanjem (Stripe). Sistem automatski prepoznaje uplatu i istog trenutka generiše startni broj i potvrdu.
- **User-Centric Profiling:** Svaki trkač ima svoj profil sa istorijom rezultata, kategorijama i digitalnim kartonom, dok organizatori imaju dashboard za upravljanje logistikom trke u realnom vremenu.
- **Procesiranje rezultata na terenu:** Digitalizacija unosa vremena sa prepreka, čime se izbegava ručno sabiranje i omogućava trenutni uvid u rang listu.

2. Prikupljanje korisničkih zahteva

2.1 „User story” analiza

Identifikacija aktera

- **Administrator:** Vrhovni korisnik zadužen za upravljanje sistemom, verifikaciju naloga organizatora i uvid u globalnu analitiku.
- **Organizator:** Korisnik koji kreira trke, definiše staze i prepreke, te upravlja prijavama korisnika na trku.
- **Takmičar:** Registrovan korisnik koji se prijavljuje na trke, vrši uplate i prati svoje rezultate.
- **Neregistrovani posetilac:** Gost koji pretražuje javno dostupne informacije o trkama i rezultatima.

User Story	Domen
Kao Administrator, želim da verifikujem profil organizatora trke, kako bih osigurao kredibilitet trka na platformi.	User Domain
Kao Organizator, želim da definišem težinu i redosled prepreka na stazi, kako bih takmičarima pružio precizne informacije.	Race Domain
Kao Takmičar, želim da izvršim uplatu preko Stripe platforme, kako bih automatski rezervisao startni broj.	Payment Domain
Kao Neregistrovani posetilac, želim da pretražim trke po lokaciji, kako bih našao najbliži događaj za koji sam zainteresovan.	Race Domain

2.2 Specifikacija funkcionalnih zahteva

Upravljanje korisnicima (User Domain):

- Sistem mora omogućiti Administratoru uvid u profil organizatora pre nego što mu dodeli status "Verifikovan".
- Sistem mora automatski onemogućiti kreiranje trka korisnicima koji nemaju ulogu verifikovanog Organizatora.
- Sistem mora podržati suspenziju (ban) naloga takmičara ili organizatora u slučaju kršenja pravila platforme.

Konfiguracija trka i staza (Race Domain):

- Sistem mora omogućiti Organizatoru definisanje parametara staze, uključujući distancu, visinsku razliku i nivo težine.
- Sistem mora pružiti interfejs za vizuelno ređanje prepreka iz kataloga na određenu stazu, uz dodelu rednog broja svakoj prepreci.
- Sistem mora podržati filtriranje i pretragu trka prema lokaciji, dužini trke i težini prepreka za neregistrovane posetioce.

Proces uplate (Payment Domain):

- Sistem mora generisati Stripe Checkout sesiju za Takmičara i vratiti validan URL za preusmeravanje na plaćanje.
- Sistem mora asinhrono obraditi Stripe Webhook pozive kako bi potvrdio status transakcije i automatski ažurirao status prijave Takmičara.
- Sistem mora automatski dodeliti startni broj (Bib number) Takmičaru nakon što sistem primi potvrdu o uspešnoj uplati.

2.3 Specifikacija nefunkcionalnih zahteva

Atribut kvaliteta	Projektovana vrednost i jedinica mere	Opis zahteva
Dostupnost (Availability)	99.9% (vremena na godišnjem nivou)	Sistem mora biti dostupan takmičarima za pregled trka i prijave 24/7. Maksimalno dozvoljeno vreme neraspoloživosti sistema je oko 9 sati godišnje.
Skaliranje (Scalability)	Do 5.000 (konkurentnih korisnika)	Sistem mora podržati do 5.000 istovremenih korisnika na platformi u trenucima otvaranja prijave za popularne trke bez pada performansi.
Konzistentnost podataka	Do 1.000 ms (milisekundi)	U mikroservisnoj arhitekturi, podatak o uspešnoj uplati iz Payment servisa mora biti vidljiv u Race servisu (status prijave) u roku od jedne sekunde.

3. Projektovanje arhitekture aplikacije

3.1 Software Architecture Canvas

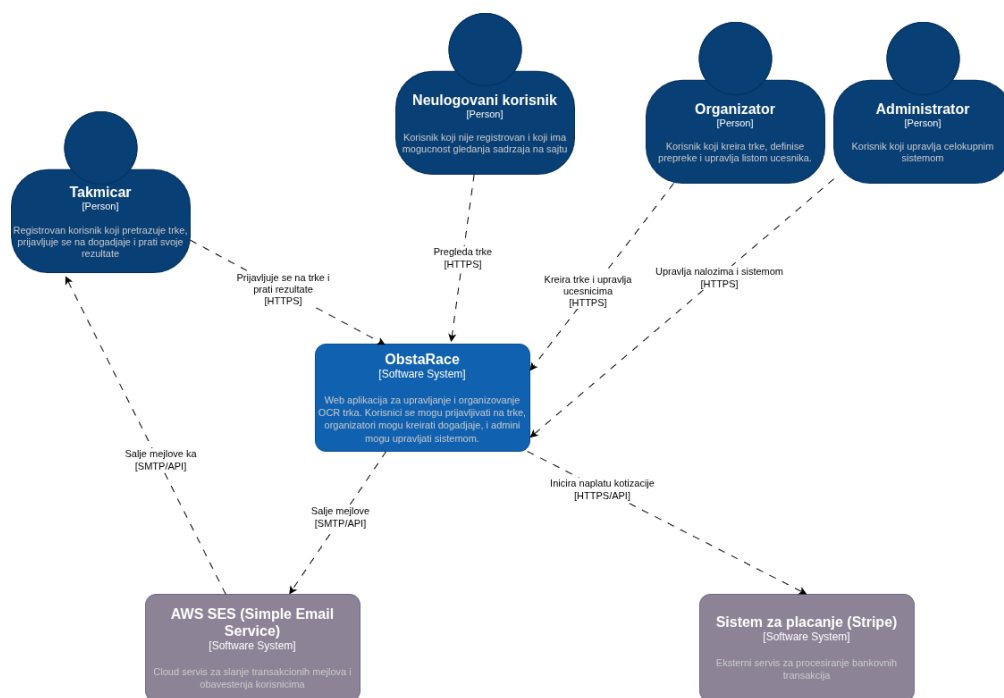
Softverski sistem: ObstaRace	Projektantski tim: Vukašin Kovačević Sofija Turina	Datum: 18.04.2026	Iteracija: 1.0
Poslovni problem (Business Case) Digitalizacija procesa organizacije OCR trka. Trenutni problem je manuelno vođenje prijava, neautomatizovana naplata i nepreglednost rezultata. Sistem treba da omogući laku prijavu takmičarima i efikasno upravljanje resursima (stazama i preprekama) organizatorima.			
Pregled funkcionalnih zahteva: - Upravljanje korisničkim nalogima i verifikacija organizatora. - Konfiguracija trka sa dodavanjem prepreka na staze. - Automatizacija procesa plaćanja kotizacija putem Stripe platforme. - Evidencija rezultata trke i generisanje rang liste u realnom vremenu.		Kontekst - Korisnici: Takmičari, Organizatori, Administratori. - Eksterni sistemi: Stripe API (plaćanje), AWS SES (mejlovi), AWS S3 (skladištenje fajlova). - Baze podataka: PostgreSQL (relacioni podaci), MongoDB (rezultati).	
Organizaciona ograničenja - Time-to-Market: Prva verzija mora biti gotova pre jeka sezone trka kako bismo uhvatili talas prijava. - Ograničen inženjerski kadar: Tim nema resurse da razvija sopstvena rešenja za plaćanje i bezbednost, pa smo primorani da zavisimo od eksternih API-ja (Stripe, AWS). - Budžetski limit: Troškovi infrastrukture moraju ostati u okviru "free tier" paketa dok se ne dostigne određeni broj prijava, što ograničava izbor servisa.		Tehnička ograničenja - Zavisnost od eksternih API-ja: Sistem je kritično zavisian od dostupnosti Stripe i AWS servisa. Ako oni padnu, naš proces registracije prestaje da radi. - Memorijski limit (Streaming): Veliki izveštaji i rang liste se moraju procesirati asinhrono kroz yield, kako bi se sprečio "crash" servera usled preopterećenja RAM-a. - Race Condition (Concurrency): Sistem mora da spreči duplu dodelu poslednjeg slobodnog mesta na trci u situacijama kada više korisnika plaća u istoj milisekundi.	
Atributi kvaliteta: - Dostupnost: 99.9% (Sistem mora biti operativan tokom celog perioda prijava). - Konzistentnost: Eventualna konzistentnost (sinhronizacija uplate i registracije < 1s). - Skalabilnost: Podrška za 5.000 konkurentnih korisnika tokom "peak" perioda.			
Hipoteze arhitekture softvera - H1: Mikroservisna dekompozicija će omogućiti nezavisno skaliranje. Opravdanje: Pretpostavljamo da će servis za rezultate biti pod mnogo većim opterećenjem nego servis za registraciju korisnika. Rizik/Trošak: Ako se ispostavi da je saobraćaj ravnomeran, arhitektura postaje nepotrebno složena, što povećava cenu održavanja i infrastrukture. - H2: Upotreba MongoDB-a za rang liste će smanjiti latenciju. Opravdanje: Nestruktuirani podaci o prolaznim vremenima se brže čitaju iz NoSQL baze nego kroz kompleksne Join-ove u SQL-u. Rizik/Trošak: Uvođenje druge baze (Polyglot persistence) povećava troškove memorije na serveru i komplikuje backup procedure. - H3: Asinhrono strimovanje (yield) će sprečiti memorijsko zagušenje. Opravdanje: Pretpostavljamo da će izveštaji sa hiljadama takmičara biti česti. Rizik/Trošak: Implementacija		Tehnički izazovi i rizici - Dual-DB Konzistentnost: Teško je održati SQL (uplate) i NoSQL (rezultati) baze uvek usklađenim bez klasičnih transakcija. - Race Condition: Rizik da dva korisnika se prijave na poslednje mesto na trci u istoj milisekundi (problem konkurentnosti). - Zavisnost od API-ja: Ako Stripe ili AWS padnu, naš sistem postaje potpuno neupotrebljiv (kritičan rizik). - SQL Injection (SQLi): Rizik od neovlašćenog upada u bazu putem malicioznih unosa u forme.	

asinhronih generatora je kompleksnija za debugovanje, što može produžiti vreme razvoja (vreme = novac).

- H4: Oslanjanje na Stripe Webhook-ove obezbeđuje tačnost plaćanja. Opravdanje: Verujemo eksternom provajderu više nego sopstvenoj implementaciji provere transakcija. Rizik/Trošak: Ako Stripe API ima prekid, naš sistem postaje neupotrebljiv za prijave, što direktno vodi do gubitka prihoda organizatora i nas samih.

3.2 Arhitektura aplikacije

C4 Diagram – Level 1:



C4 Model – Level 1 (System Context)

Ovaj dijagram prikazuje ObstaRace sistem kao "crnu kutiju" i definiše njegove granice prema spoljnom svetu.

1. Akteri (Korisnici):

- **Takmičar:** Registrovani korisnik koji pretražuje trke, vrši prijave i prati svoje rezultate.
- **Organizator:** Kreira događaje (trke), definiše prepreke i upravlja listama učesnika.
- **Administrator:** Vršni nadzor nad celim sistemom, upravlja nalogima i rešava sporne situacije.
- **Neulogovani korisnik:** Posetilac koji može samo da pregleda javno dostupne informacije o trkama.

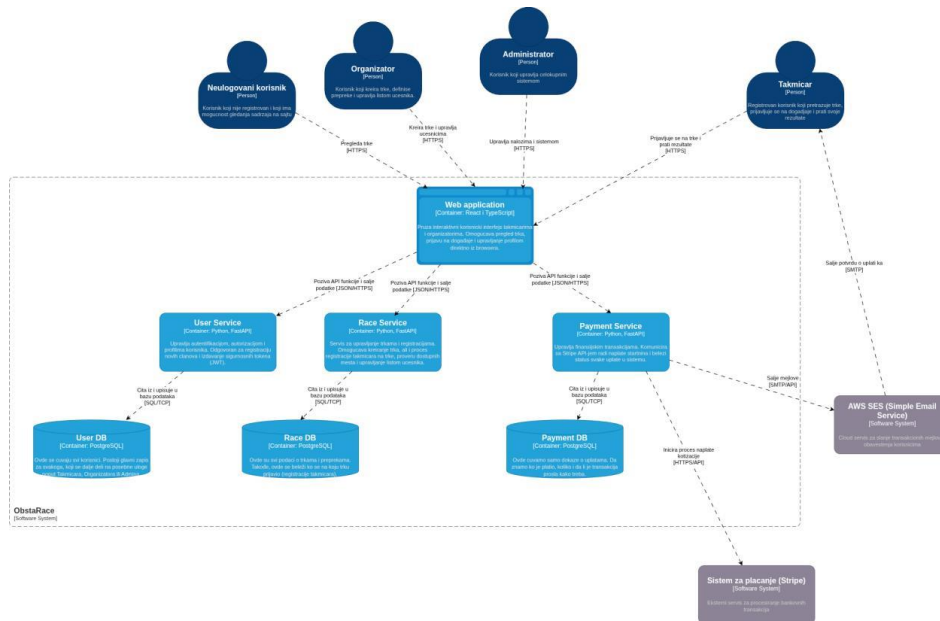
2. Centralni sistem (ObstaRace):

- Predstavlja web aplikaciju koja objedinjuje poslovnu logiku za OCR trke, upravljanje korisnicima i obradu podataka.

3. Eksterni sistemi (Integracije):

- **Stripe (Sistem za plaćanje):** Eksterni servis zadužen za bezbedno procesiranje bankovnih transakcija (kotizacija).
- **AWS SES (Email Service):** Servis za slanje transakcionih mejlova (potvrde o prijavi, zaboravljene lozinke).

C4 Diagram – Level 2:

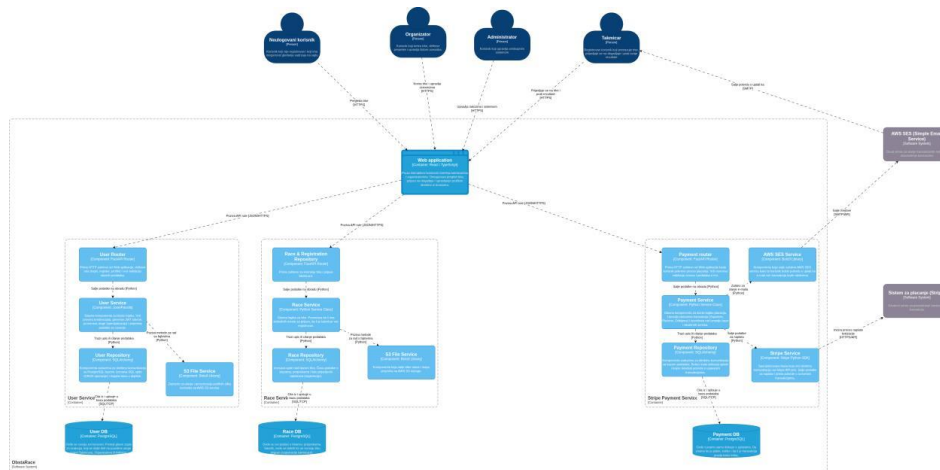


C4 Model – Level 2 (Container Diagram)

Ovaj dijagram prikazuje unutrašnju arhitekturu ObstaRace sistema, zasnovanu na mikroservisima.

- **Web Application (React & TypeScript):** Front-end koji korisnici vide u browseru. On komunicira sa svim backend servisima preko HTTPS protokola.
- **Backend Mikroservisi (FastAPI):**
- **User Service:** Upravlja loginom (JWT) i profilima (koristi PostgreSQL).
- **Race Service:** Srce sistema za kreiranje trka i registraciju takmičara (koristi PostgreSQL).
- **Payment Service:** Integriše se sa Stripe-om i prati status uplata (koristi PostgreSQL).
- **Data Layer:** Primenjen je princip Database-per-Service, što omogućava nezavisno skaliranje i korišćenje različitih tehnologija baze (SQL i NoSQL) tamo gde najviše odgovaraju.
- **Eksterne integracije:** Servisi direktno komuniciraju sa AWS SES (za mejlove) i Stripe (za pare).

C4 Diagram – Level 3:

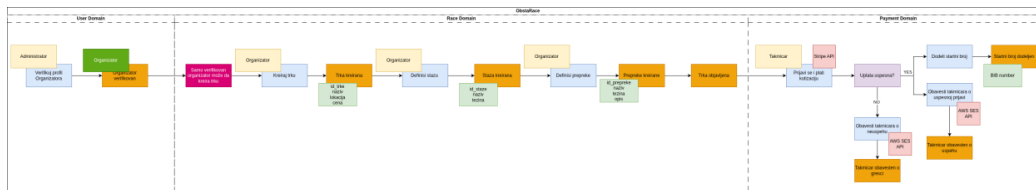


C4 Model – Level 3 (Component Diagram)

Ovaj dijagram dekomponuje kontejnere na logičke komponente unutar svakog mikroservisa. Arhitektura prati Layered (slojevitu) strukturu radi lakšeg održavanja:

- **API Routers (Controllers):** Ulazne tačke koje prihvataju HTTP zahteve sa frontenda, vrše osnovnu validaciju (Pydantic) i prosleđuju ih dalje.
- **Service Layer (Business Logic):** Srce svakog servisa gde se izvršavaju poslovna pravila (npr. provera da li je trka popunjena, računanje rang liste).
- **Repository Layer (Data Access):** Komponente zadužene isključivo za komunikaciju sa bazama podataka (PostgreSQL/MongoDB) koristeći ORM (SQLAlchemy).
- **Infrastructure Adapters (External Services):**
- **Stripe Service:** Komponenta koja enkapsulira pozive ka Stripe API-ju.
- **SES Mail Service:** Komponenta za asinhrono slanje emailova.

3.3 EventStorming metodologija



EventStorming dijagram procesa

Na osnovu definisanih korisničkih priča i funkcionalnih zahteva, izvršena je dekompozicija sistema pomoću EventStorming metodologije. Proces je podeljen na tri ključna domena (Bounded Contexts) koji komuniciraju asinhrono putem događaja.

Legenda elemenata:

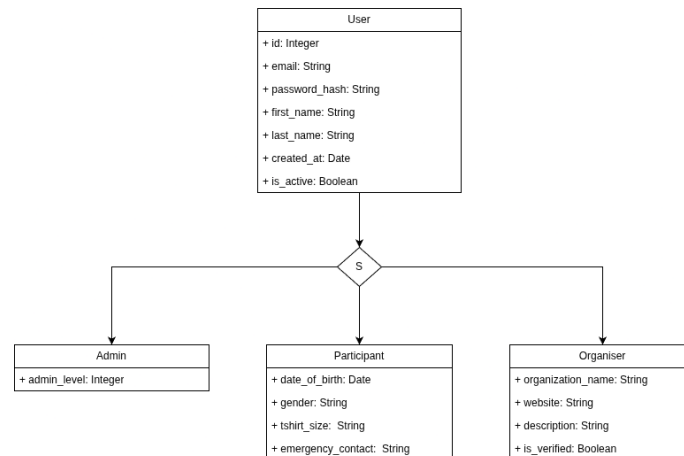
- **Akter (Žuto):** Korisnik koji inicira akciju (Administrator, Organizator, Takmičar).
- **Komanda (Plavo):** Akcija ili zahtev poslat sistemu (npr. Kreiraj trku, Izvrši uplatu).
- **Događaj (Narandžasto):** Ključna promena stanja u sistemu (npr. Trka objavljena, Uplata potvrđena).
- **Agregat/Artefakt (Zeleno):** Entiteti koji čuvaju stanje i prateći podaci (npr. Trka, Staza, ID uplate).
- **Poslovna polisa (Ljubičasto):** Logička pravila koja okidaju dalje akcije (npr. Verifikacija statusa organizatora).
- **Eksterni sistemi (Roze):** Spoljni servisi integrisani u proces (Stripe API za plaćanje i AWS SES za notifikacije).

Opis procesnih tokova po domenima:

- **User Domain (Upravljanje korisnicima):** Proces počinje inicijalnom proverom profila od strane Administratora. Tek nakon što se generiše događaj „Profil organizatora verifikovan“, sistem dozvoljava prelazak u sledeću fazu. Ovim je osiguran integritet i kredibilitet platforme.
- **Race Domain (Konfiguracija trka):** Verifikovani Organizator kreira trku, definiše parametre staze i vizuelno reda prepreke. Svaka faza (kreiranje trke, staze i prepreka) generiše svoj Artefakt sa ključnim metapodacima. Proces u ovom domenu se završava događajem „Trka objavljena“, čime ona postaje dostupna za pretragu i prijave.
- **Payment Domain (Proces uplate i prijavljivanja):** Takmičar pokreće proces plaćanja koji se oslanja na Stripe API. Sistem asinhrono obrađuje Stripe Webhook pozive. Ukoliko je polisa provere uplate uspešna (YES grana), sistem automatski dodeljuje Startni broj (Bib number) i putem AWS SES servisa šalje potvrdu takmičaru. U slučaju neuspeha (NO grana), takmičar biva obavešten o grešci.

3.4 Definicija modela podataka

User UML Class Diagram (PostgreSQL):

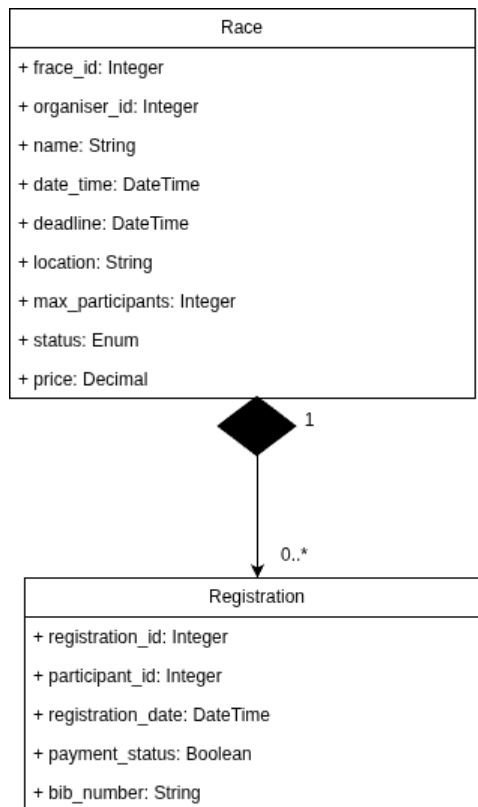


User UML dijagram klasa

Dijagram prikazuje konceptualni model korisnika zasnovan na mehanizmu nasleđivanja (specijalizacije).

- **Superklasa (User):** Sadrži zajedničke atribute za sve korisnike sistema, kao što su kredencijali (email, password_hash), osnovni podaci (first_name, last_name) i status naloga.
- **Specijalizacija (S):** Čvor koji označava podelu na specifične uloge. U PostgreSQL-u se ovo najčešće implementira preko Table-per-Type strategije (svaka podklasa je posebna tabela povezana stranim ključem sa User tabelom).
- **Podklase:**
 - **Admin:** Čuva specifične podatke o nivou administratorskih prava (admin_level).
 - **Participant (Takmičar):** Sadrži podatke neophodne za OCR trke: datum rođenja, pol, veličinu majice i kontakt za hitne slučajeve.
 - **Organiser (Organizator):** Čuva poslovne podatke o organizaciji, veb-sajt i status verifikacije.

Race & Registration UML Class Diagram (PostgreSQL):

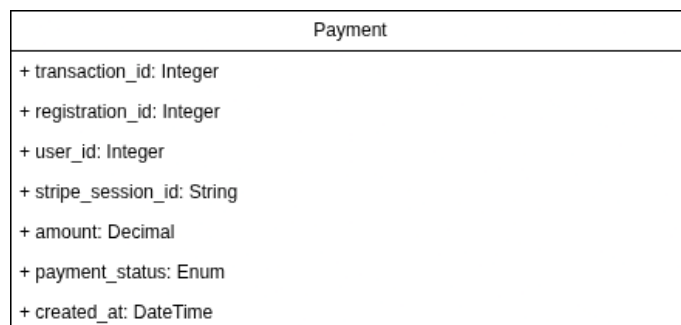


Race & Registration UML dijagram klasa

Ovaj dijagram definiše strukturu trka, staza i procesa registracije. Fokus je na kompoziciji i relacijama koje omogućavaju fleksibilnost događaja.

- **Race (Trka):** Centralna klasa koja sadrži opšte informacije (ime, lokacija, kotizacija, limit takmičara). Koristi kompoziciju (crni romb) ka klasi Registration, što znači da ovaj entitet ne može postojati bez same trke.
- **Registration (Registracija):** Beleži svaku pojedinačnu prijavu takmičara. Sadrži payment_status (bitno za integraciju sa Stripe-om) i bib_number (startni broj).

Payment UML Class Diagram (PostgreSQL):



Payment UML dijagram klasa

Klasa Payment predstavlja logički zapis finansijske transakcije unutar sistema. Njena primarna uloga je praćenje toka novca i povezivanje eksternog procesora plaćanja sa internim procesom registracije.

- **Identifikatori i veze:** Pored sopstvenog transaction_id, klasa čuva registration_id i user_id kao strane ključeve (FK) kako bismo u svakom trenutku znali ko je platio i za koju konkretnu prijavu.
- **stripe_session_id:** Ključni atribut koji čuva jedinstveni identifikator sesije sa Stripe platforme. Ovo nam omogućava da putem Webhook-ova uparimo potvrdu o uplati sa našim zapisom u bazi.
- **payment_status (Enum):** Prati životni ciklus uplate (npr. Pending, Succeeded, Failed, Refunded).
- **amount:** Iznos uplate, definisan kao Decimal radi preciznosti.

Literatura i reference

<https://tekiegirl.github.io/Archangels/3.ViewsAndPerspectives/C4Models/>

<https://www.archipeg.com/learn/c4-model-v1-metamodel>